SAC（Soft Actor-Critic）完整推导与设计动机

从最大熵强化学习到连续控制实现·17 节完整教程

强化学习技术教程

2026-10-15

目录

1. 从标准强化学习开始 **2**

2. 第一步：最大熵强化学习 **3**
2.1 策略熵与温度参数 . . . 3
2.2 把熵写成可采样的形式 . . . 3

3. 第二步：Soft Bellman 方程 **4**
3.1 与普通 Bellman 方程对比 . . . 4
3.2 定义 Soft V 和 Soft Q . . . 4
3.3 Soft Bellman Expectation Equation . . . 4

4. 第三步：Soft Policy Improvement **5**
4.1 为什么普通贪心改进不够? . . . 5
4.2 变分法求 Boltzmann Policy . . . 5

5. 第四步：把策略改进转为 KL 最小化 **6**
5.1 计算 Boltzmann 分布为什么困难? . . . 6
5.2 为什么精确 Soft Policy Iteration 可以改进策略? . . . 6

6. 第五步：从 Soft Policy Iteration 到 Actor-Critic **7**

7. 第六步：为什么能够 Off-Policy 学习? **7**
7.1 PPO 的数据分布依赖 . . . 7
7.2 Bellman 更新为何可以使用旧经验? . . . 7

8. 第七步：完整推导 Critic Loss **8**
8.1 从 Soft Bellman Target 到 TD Regression . . . 8
8.2 Target Network：减缓目标漂移 . . . 8
8.3 Clipped Double Q：缓解价值高估 . . . 8
8.4 最终 Critic Target 和 Loss . . . 9

9. 第八步：完整推导 Actor Loss **9**

10. 第九步：Reparameterization Trick **10**
10.1 把依赖参数的随机采样变成可微函数 . . . 10
10.2 Tanh-Squashed Gaussian Policy . . . 10
10.3 把 Actor Loss 改写为 Pathwise Objective . . . 10

11. 第十步：Tanh Gaussian 的 Log Probability 修正 **11**

12. 第十一步：Automatic Entropy Tuning **12**
12.1 从固定温度到熵约束 . . . 12

12.2 温度损失及其更新方向 . . . 12
12.3 为什么学习 Log-Alpha? . . . 12
12.4 Target Entropy 的经验选择 . . . 13

13. 第十二步：完整的现代 SAC 算法 **13**
13.1 模块与三个优化问题 . . . 13
13.2 一次更新的完整步骤 . . . 13
13.3 教学用伪代码 . . . 14

14. SAC v1 与现代 SAC 的区别 **15**
14.1 原始 SAC v1：显式训练 V 网络 . . . 15
14.2 现代 SAC：去掉显式 V 网络 . . . 15

15. SAC 与 PPO 的本质区别 **15**
15.1 策略更新公式对比 . . . 15
15.2 对比表 . . . 16
15.3 为什么机器人仿真中 PPO 仍常见? . . . 16

16. 对灵巧手与 Flow Matching Policy 的启示 **16**
16.1 机器人连续动作的局部优化 . . . 16
16.2 冻结预训练动作生成器，优化输入噪声 . . . 16
16.3 为什么 Flow Matching Policy 直接接 SAC 更困难? . . . 17

17. 最终总结：五组关键公式与理解框架 **17**
最关键的 PPO / SAC 区别 . . . 18
参考资料 . . . 18

阅读说明：本教程以现代连续动作 SAC（两个 Q Critic、无单独 V 网络、可自动调节温度）为主线，完整介绍理论动机、数学推导、工程实现及与 PPO、机器人操作、Flow Matching 策略的联系。第 14 节单独说明原始 SAC v1。文中符号在全文保持一致。

1. 从标准强化学习开始

SAC（Soft Actor-Critic）是一种面向连续控制的 **Off-Policy、随机策略、Actor-Critic** 强化学习算法。其基本思路是：在最大化长期回报的同时鼓励策略维持适当的随机性，借助 Soft Bellman 递推学习动作价值，再用可微的 Q 函数优化 Actor。其主要组成包括最大熵目标、Soft Policy Iteration、Replay Buffer、双 Q 网络、目标网络、重参数化技巧和自动熵温度调节。

考虑折扣型 MDP：

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma), \qquad 0 \le \gamma < 1.$$

其中 $s_t \in \mathcal{S}$ 是状态，$a_t \in \mathcal{A}$ 是动作，$P(s_{t+1} \mid s_t, a_t)$ 是环境转移核，$r(s_t, a_t)$ 是即时奖励，$\pi_\theta(a \mid s)$ 是参数为 θ 的策略。标准强化学习最大化：

$$\boxed{J_{\mathrm{RL}}(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\right].}$$

动机问题：只优化奖励为什么可能不够? 以机器人抓取为例，假设当前 Critic 对三个动作给出如下估计：

候选动作	Critic 估计的 Q 值
从左侧抓取	10
从右侧抓取	9.8

候选动作	Critic 估计的 Q 值
从上方抓取	9.5

如果策略立即集中于左侧抓取，就可能放弃探索。更严重的是，这些估计可能不准确：例如当前 Critic 给出 $\widehat{Q}(s, a_{\text{left}}) = 10$、$\widehat{Q}(s, a_{\text{right}}) = 8$，但真实值却是 $Q(s, a_{\text{left}}) = 8$、$Q(s, a_{\text{right}}) = 10$。由于右侧抓取很少被尝试，估计错误可能长期无法纠正。

于是我们希望策略不只是选择当前估计的最佳动作，还可以在高价值动作附近保持适度的随机性。这就是最大熵强化学习（Maximum Entropy RL）的出发点。

2. 第一步：最大熵强化学习

2.1 策略熵与温度参数

离散动作的 Shannon 熵是 $\mathcal{H}(\pi(\cdot \mid s)) = -\sum_a \pi(a \mid s) \log \pi(a \mid s)$；连续动作通常使用微分熵：

$$\boxed{\mathcal{H}(\pi(\cdot \mid s)) = -\int \pi(a \mid s) \log \pi(a \mid s)\, da = \mathbb{E}_{a\sim\pi}[-\log \pi(a \mid s)].}$$

对一维正态策略 $\pi(a \mid s) = \mathcal{N}(\mu(s), \sigma^2(s))$：

$$\mathcal{H}(\pi) = \frac{1}{2}\log(2\pi e\sigma^2) = \log\sigma + \frac{1}{2}\log(2\pi e).$$

因此，方差越大，熵越大，动作采样也越分散。**注意**：连续分布的微分熵可以为负，而且依赖动作变量的尺度；这与离散熵不同。

引入温度 $\alpha \geq 0$，将目标改为：

$$\boxed{J_{\text{ME}}(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty} \gamma^t \big(r(s_t, a_t) + \alpha\mathcal{H}(\pi(\cdot \mid s_t))\big)\right].}$$

α 控制高回报与高熵之间的权衡：$\alpha \to 0$ 时趋于普通 RL；α 较大时鼓励更分散的策略。需要单独的系数，是因为奖励与熵的数值尺度可能完全不同。例如 $r \approx 100$，而 $\mathcal{H}(\pi) \approx 5$，若直接相加，熵项可能影响很小；通过 α 才能控制相对权重。

2.2 把熵写成可采样的形式

由 $\mathcal{H}(\pi(\cdot \mid s)) = \mathbb{E}_{a\sim\pi}[-\log \pi(a \mid s)]$，有：

$$\boxed{J_{\text{ME}}(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty} \gamma^t \big(r(s_t, a_t) - \alpha \log \pi(a_t \mid s_t)\big)\right].} \tag{1}$$

可以将 $\tilde{r}_t = r_t - \alpha \log \pi(a_t \mid s_t)$ 看成带熵正则的奖励表达。它依赖当前策略，**不是**与策略无关的固定环境奖励。改写的关键好处是：连续动作算法只需采样 $a \sim \pi$ 并计算 $\log \pi(a \mid s)$，而不必对整个动作空间显式积分求熵。

3. 第二步：Soft Bellman 方程

3.1 与普通 Bellman 方程对比

标准价值函数满足：

$$V^\pi(s) = \mathbb{E}_{a\sim\pi}[Q^\pi(s,a)], \quad Q^\pi(s,a) = \mathbb{E}_{s'\sim P}[r(s,a) + \gamma V^\pi(s')].$$

将两式合并：

$$Q^\pi(s,a) = \mathbb{E}_{s',a'}[r(s,a) + \gamma Q^\pi(s',a')].$$

最大熵 RL 需要把未来策略的熵也纳入价值计算。

3.2 定义 Soft V 和 Soft Q

采用 SAC 常用的一种一致约定：

$$\boxed{V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty}\gamma^t(r_t - \alpha\log\pi(a_t \mid s_t)) \,\middle|\, s_0 = s\right],}$$

$$\boxed{Q^\pi(s,a) = \mathbb{E}_\pi\left[r_0 + \sum_{t=1}^{\infty}\gamma^t(r_t - \alpha\log\pi(a_t \mid s_t)) \,\middle|\, s_0 = s, a_0 = a\right].}$$

区别是：V^π 包含当前动作的熵项；Q^π 固定当前动作，因此把熵项从下一步开始计入。这种约定便于书写递推。由条件期望：

$$\boxed{V^\pi(s) = \mathbb{E}_{a\sim\pi}\big[Q^\pi(s,a) - \alpha\log\pi(a \mid s)\big].} \tag{2}$$

等价地：

$$\boxed{V^\pi(s) = \mathbb{E}_{a\sim\pi}[Q^\pi(s,a)] + \alpha\mathcal{H}(\pi(\cdot \mid s)).} \tag{3}$$

普通 V 衡量当前策略未来可获得的奖励；Soft V 同时衡量奖励与可保持的随机性。

3.3 Soft Bellman Expectation Equation

由定义和马尔可夫性质：

$$Q^\pi(s,a) = \mathbb{E}_{s'\sim P}[r(s,a) + \gamma V^\pi(s')].$$

代入式 (2)：

$$\boxed{Q^\pi(s,a) = \mathbb{E}_{\substack{s'\sim P(\cdot|s,a)\\ a'\sim\pi(\cdot|s')}}\left[r(s,a) + \gamma\left(Q^\pi(s',a') - \alpha\log\pi(a' \mid s')\right)\right].} \tag{4}$$

与普通 Bellman 方程相比，增加了 $-\alpha\log\pi(a' \mid s')$。这正是熵奖励通过下一状态的 Soft Value 进入 Critic Target 的方式。该公式给出了 SAC **策略评估** 的理论基础，但还需要推导策略改进。

4. 第三步：Soft Policy Improvement

4.1 为什么普通贪心改进不够?

普通策略迭代使用 $\pi_{\text{new}}(s) \in \arg\max_a Q^{\pi_{\text{old}}}(s,a)$。随机策略若只最大化 $\mathbb{E}_a Q$，最优解也可能退化为集中于最大 Q 动作的确定性分布。SAC 希望在改进动作价值的同时保留熵。

固定状态 s 和旧策略的 $Q_{\text{old}}(s,a) = Q^{\pi_{\text{old}}}(s,a)$，考虑:

$$\boxed{\pi_{\text{new}} = \arg\max_{\pi} \left\{ \mathbb{E}_{a\sim\pi}[Q_{\text{old}}(s,a)] + \alpha \mathcal{H}(\pi(\cdot \mid s)) \right\}.} \tag{5}$$

即最大化泛函:

$$F(\pi) = \int \pi(a \mid s) Q_{\text{old}}(s,a)\, da - \alpha \int \pi(a \mid s) \log \pi(a \mid s)\, da,$$

并约束 $\int \pi(a \mid s)\, da = 1$。

4.2 变分法求 Boltzmann Policy

设 $\alpha > 0$，并假定归一化常数有限。拉格朗日函数为:

$$\begin{aligned} \mathcal{L}(\pi, \lambda) = & \int \pi(a \mid s) Q_{\text{old}}(s,a)\, da \\ & - \alpha \int \pi(a \mid s) \log \pi(a \mid s)\, da \\ & + \lambda \left(\int \pi(a \mid s)\, da - 1 \right). \end{aligned}$$

对密度的变分导数:

$$\frac{\delta \mathcal{L}}{\delta \pi(a \mid s)} = Q_{\text{old}}(s,a) - \alpha(\log \pi(a \mid s) + 1) + \lambda.$$

令其为零:

$$\log \pi(a \mid s) = \frac{Q_{\text{old}}(s,a)}{\alpha} + \frac{\lambda}{\alpha} - 1.$$

取指数并用积分归一化，得到:

$$\boxed{\pi^*_{\text{new}}(a \mid s) = \frac{\exp(Q_{\text{old}}(s,a)/\alpha)}{Z_{\text{old}}(s)}, \quad Z_{\text{old}}(s) = \int_{\mathcal{A}} \exp(Q_{\text{old}}(s,a')/\alpha)\, da'.} \tag{6}$$

它是 **Boltzmann Policy**：Q 越大概率越大，α 越大分布越平缓，$\alpha \to 0^+$ 时概率集中于 Q 最大的动作。假如两个动作 $Q(a_1) = 2, Q(a_2) = 1$，则 $\alpha = 0.5$ 时 $\pi(a_1) \approx 0.881, \pi(a_2) \approx 0.119$；$\alpha = 2$ 时变为 $0.622, 0.378$。

动机总结：最大熵策略改进不再是简单地选择 $\arg\max_a Q$，而是学习一个由价值函数决定的高价值动作分布。

5. 第四步：把策略改进转为 KL 最小化

5.1 计算 Boltzmann 分布为什么困难?

对高维连续动作，$Z(s) = \int e^{Q(s,a)/\alpha}\, da$ 通常无法高效计算，因而不能直接构造或采样归一化后的理想策略。SAC 通过参数化 Actor π_θ，让它 **投影到 Boltzmann 分布**，并消去 $Z(s)$。
令:

$$\pi_B(a \mid s) = \frac{\exp(Q_{\text{old}}(s,a)/\alpha)}{Z(s)}.$$

考虑前向方向为 $D_{\text{KL}}(\pi_\theta \| \pi_B)$ 的 KL 散度:

$$\begin{aligned} D_{\text{KL}}(\pi_\theta \| \pi_B) &= \mathbb{E}_{a\sim\pi_\theta}\left[\log \pi_\theta(a \mid s) - \log \pi_B(a \mid s)\right] \\ &= \mathbb{E}_{a\sim\pi_\theta}\left[\log \pi_\theta(a \mid s) - \frac{Q_{\text{old}}(s,a)}{\alpha}\right] + \log Z(s). \end{aligned}$$

Q_{old} 固定，$\log Z(s)$ 与 Actor 参数无关，于是:

$$\boxed{\arg\min_\theta D_{\text{KL}}(\pi_\theta \| \pi_B) = \arg\min_\theta \mathbb{E}_{a\sim\pi_\theta}\left[\alpha \log \pi_\theta(a \mid s) - Q_{\text{old}}(s,a)\right].} \tag{7}$$

这里便得到了 SAC 的 **Actor Loss 原型**：既无需计算 $Z(s)$，也不用对高维动作空间枚举积分。
更进一步:

$$\boxed{\mathbb{E}_{a\sim\pi_\theta}[Q(s,a)] + \alpha\mathcal{H}(\pi_\theta(\cdot \mid s)) = \alpha \log Z(s) - \alpha D_{\text{KL}}(\pi_\theta \| \pi_B).} \tag{8}$$

这表明：最大化 Soft Value 与逼近 Boltzmann Policy 是同一优化问题的两种表述。

5.2 为什么精确 Soft Policy Iteration 可以改进策略?

设 π_{old} 在允许的策略族中，且在每个状态上准确求解 KL 投影。根据最优性:

$$D_{\text{KL}}(\pi_{\text{new}} \| \pi_B) \le D_{\text{KL}}(\pi_{\text{old}} \| \pi_B).$$

使用式 (8) 得到:

$$\boxed{\mathbb{E}_{a\sim\pi_{\text{new}}}[Q^{\pi_{\text{old}}}(s,a) - \alpha \log \pi_{\text{new}}(a \mid s)] \ge V^{\pi_{\text{old}}}(s).} \tag{9}$$

定义固定策略 π 的 Soft Bellman 算子:

$$(\mathcal{T}^\pi Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s',a'\sim\pi}\left[Q(s',a') - \alpha \log \pi(a' \mid s')\right].$$

式 (9) 结合 Bellman 方程可得:

$$Q^{\pi_{\text{old}}} \le \mathcal{T}^{\pi_{\text{new}}} Q^{\pi_{\text{old}}}.$$

$\mathcal{T}^\pi$ 是单调算子，在适当的有界性假设下还满足压缩性质:

$$\|\mathcal{T}^\pi Q_1 - \mathcal{T}^\pi Q_2\|_\infty \le \gamma \|Q_1 - Q_2\|_\infty.$$

反复应用 $\mathcal{T}^{\pi_{\text{new}}}$：

$$Q^{\pi_{\text{old}}} \leq \mathcal{T}^{\pi_{\text{new}}} Q^{\pi_{\text{old}}} \leq \cdots \longrightarrow Q^{\pi_{\text{new}}}.$$

因此：

$$\boxed{Q^{\pi_{\text{new}}}(s,a) \geq Q^{\pi_{\text{old}}}(s,a).} \tag{10}$$

这说明精确的 **Soft Policy Evaluation + Soft Policy Improvement** 具有策略改进性质。**边界条件**：这里是精确评估与精确投影下的理论结论；神经网络、有限采样、随机梯度和函数逼近的实际 SAC 并不因此获得全局收敛保证。

6. 第五步：从 Soft Policy Iteration 到 Actor-Critic

至此得到了两步：

- **Policy Evaluation**：根据 $Q^{\pi}(s,a) = \mathbb{E}[r + \gamma(Q^{\pi}(s',a') - \alpha \log \pi(a' \mid s'))]$ 评估策略。
- **Policy Improvement**：根据 $\min_{\pi} \mathbb{E}_{a\sim\pi}[\alpha \log \pi(a \mid s) - Q(s,a)]$ 改进策略。

连续状态和动作无法用表格存储完整的 Q 和 π，因此用神经网络近似：

$$\boxed{\text{Actor: } \pi_\theta(a \mid s), \qquad \text{Critic: } Q_\phi(s,a).}$$

Actor 输入状态，输出动作分布；Critic 输入状态与动作，输出标量 Q 值。由 Soft Bellman 方程构造 TD 误差：

$$\delta = Q_\phi(s,a) - \left[r + \gamma \left(Q_\phi(s',a') - \alpha \log \pi_\theta(a' \mid s')\right)\right].$$

自然的 Critic 损失为 $\frac{1}{2}\mathbb{E}[\delta^2]$；Actor 则最小化 $\mathbb{E}[\alpha \log \pi_\theta(a \mid s) - Q_\phi(s,a)]$。这里的奖励信号通过 $r \to Q_\phi \to \pi_\theta$ 间接作用于 Actor，而不是直接让环境奖励反向传播。

注意：用一个不断变化的 Q_ϕ 同时预测与构造自身目标，可能造成训练不稳定；第 8 节会引入目标网络和双 Q。

7. 第六步：为什么能够 Off-Policy 学习?

7.1 PPO 的数据分布依赖

PPO 的 Clipped Objective 依赖旧策略产生的数据和概率比值：

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}.$$

因此标准 PPO 主要使用近期 On-Policy Rollouts；旧得太久的数据会造成严重分布偏移。

7.2 Bellman 更新为何可以使用旧经验?

在固定 MDP 中，环境的条件转移 $P(s' \mid s,a)$ 与**是哪一个行为策略采样到** (s,a) 无关。过去行为策略产生的样本 (s,a,r,s') 仍是有效的环境转移样本。只需在下一个状态 s'，使用**当前策略**重新采样 $a' \sim \pi_\theta(\cdot \mid s')$，就能构造当前策略的 Soft Bellman Target：

$$y = r + \gamma\big(Q(s',a') - \alpha \log \pi_\theta(a' \mid s')\big).$$

变量	数据来源
s, a, r, s'	Replay Buffer 中的历史转移
a'	当前 Actor $\pi_\theta(\cdot \mid s')$ 重新采样
$\log \pi_\theta(a' \mid s')$	当前 Actor 计算

维护经验池：

$$\boxed{\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1}, d_t)\}.}$$

更新时从 $\mathcal{D}$ 随机抽取 Mini-Batch，而非必须收集一批新 Rollout。历史数据可反复复用，环境一步交互之后甚至可以做多次梯度更新。

重要限制：Off-Policy 不等于分布差异完全无害。若 Replay Buffer 对某些关键状态动作缺乏覆盖，神经网络 Q 在这些区域可能严重外推错误。有限数据、函数逼近与 Bootstrapping 的组合也可能引起不稳定，因此样本效率与可靠性之间仍需权衡。

8. 第七步：完整推导 Critic Loss

8.1 从 Soft Bellman Target 到 TD Regression

用当前策略采样 $a' \sim \pi_\theta(\cdot \mid s')$，理论单样本目标为：

$$y = r + \gamma \left[Q_\phi(s', a') - \alpha \log \pi_\theta(a' \mid s') \right].$$

对应回归目标是：

$$J_Q(\phi) = \frac{1}{2} \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \big[(Q_\phi(s, a) - y)^2 \big].$$

但这还不是完整 SAC：此时目标也使用同一个正在更新的网络，且 Actor 会倾向利用 Critic 的高估误差。

8.2 Target Network：减缓目标漂移

为 Critic 维护慢更新参数 $\bar{\phi}$，构造 $Q_{\bar{\phi}}$，并使用 Polyak Averaging：

$$\boxed{\bar{\phi} \leftarrow (1 - \tau)\bar{\phi} + \tau\phi, \qquad 0 < \tau \ll 1.} \tag{11}$$

例如 $\tau = 0.005$。此举减慢 TD Target 的变化速度，而非解决估计偏差本身。

8.3 Clipped Double Q：缓解价值高估

如果 $\widehat{Q}(s, a) = Q(s, a) + \epsilon(s, a)$，Actor 倾向选择 $\widehat{Q}$ 较大的动作，即使该动作的高估主要来自误差 ϵ。例如两动作真实值都为 10，Critic 却给出 12 和 8，Actor 会错误地偏好前者，并可能进一步放大误差。

因此训练两个独立 Critic：

$$Q_{\phi_1}(s, a), \qquad Q_{\phi_2}(s, a),$$

及对应目标网络：

$$Q_{\bar{\phi}_1}(s,a), \qquad Q_{\bar{\phi}_2}(s,a).$$

用保守的最小值构造目标：

$$\boxed{Q_{\text{target}}(s',a') = \min_{j=1,2} Q_{\bar{\phi}_j}(s',a').}$$

取最小值有助于缓解过高估计，但可能引入低估，并**不保证** Q 估计无偏。

8.4 最终 Critic Target 和 Loss

令 $d=1$ 表示真实终止（而非单纯 Time-Limit Truncation），最终 Target 为：

$$\boxed{y = r + \gamma(1-d)\left[\min_{j=1,2} Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi_\theta(a' \mid s')\right], \quad a' \sim \pi_\theta(\cdot \mid s').} \tag{12}$$

每个 Critic 分别拟合同一 Target：

$$\boxed{J_{Q_i}(\phi_i) = \frac{1}{2}\mathbb{E}_{\mathcal{D}}\left[\left(Q_{\phi_i}(s,a) - y\right)^2\right], \qquad i=1,2.} \tag{13}$$

$$\boxed{J_Q(\phi_1,\phi_2) = J_{Q_1}(\phi_1) + J_{Q_2}(\phi_2).} \tag{14}$$

执行 $\phi_i \leftarrow \phi_i - \eta_Q \nabla_{\phi_i} J_{Q_i}$。构造 Target 时必须停止目标分支梯度：$y \leftarrow \text{stopgrad}(y)$。对于**真实终止**，$d=1$ 从而 $y=r$；对于仅因最大步数被截断的非终止状态，通常仍应保留 Bootstrapping。

9. 第八步：完整推导 Actor Loss

第 5 节 KL 投影给出：

$$\pi_{\text{new}} = \arg\min_{\pi} \mathbb{E}_{a\sim\pi}\left[\alpha \log \pi(a \mid s) - Q(s,a)\right].$$

用 Actor π_θ 和 Critic Q_ϕ 近似，并从 Replay Buffer 提供状态：

$$\boxed{J_\pi(\theta) = \mathbb{E}_{\substack{s\sim\mathcal{D} \\ a\sim\pi_\theta(\cdot|s)}}\left[\alpha \log \pi_\theta(a \mid s) - Q_\phi(s,a)\right].} \tag{15}$$

使用双 Q 的常见版本：

$$\boxed{J_\pi(\theta) = \mathbb{E}_{\substack{s\sim\mathcal{D} \\ a\sim\pi_\theta(\cdot|s)}}\left[\alpha \log \pi_\theta(a \mid s) - \min_{i=1,2} Q_{\phi_i}(s,a)\right].} \tag{16}$$

也存在更新 Actor 时仅使用第一个 Critic 的实现。式 (16) 具有两个互补部分：最小化 $\alpha \log \pi_\theta$ 等效于鼓励高熵；最小化 $-Q_\phi$ 鼓励产生高价值动作。

关键区别：PPO 主要使用采样动作的 Advantage 和概率比更新策略；连续动作 SAC 则希望梯度穿过动作 a，利用 $\nabla_a Q_\phi(s,a)$ 对 Actor 提供局部方向信息。不过动作来自随机分布，普通“采样”不能直接写成可微的确定性计算，因此需要重参数化。

10. 第九步：Reparameterization Trick

10.1 把依赖参数的随机采样变成可微函数

令 Actor 输出均值和标准差:

$$\mu_\theta(s), \qquad \sigma_\theta(s).$$

令未压缩的动作:

$$u \sim \mathcal{N}(\mu_\theta(s), \mathrm{diag}(\sigma_\theta^2(s))).$$

直接描述为 $u \sim \pi_\theta$ 时，采样分布依赖于 θ。可等价改写为:

$$\boxed{u_\theta(s,\epsilon) = \mu_\theta(s) + \sigma_\theta(s) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).} \tag{17}$$

此时噪声 ϵ 与参数无关，并且 u_θ 对 θ 可微。例如 $\partial u/\partial \mu = I$、$\partial u/\partial \sigma = \mathrm{diag}(\epsilon)$。

10.2 Tanh-Squashed Gaussian Policy

真实机器人动作一般有界（如 $[-1,1]^d$），而 Gaussian 支持整个实数域。SAC 使用逐维 tanh:

$$\boxed{a_\theta(s,\epsilon) = \tanh\left(\mu_\theta(s) + \sigma_\theta(s) \odot \epsilon\right).} \tag{18}$$

对于实际动作区间 $[a_{\min,i}, a_{\max,i}]$，可以进一步做 $a_i^{\text{env}} = c_i + b_i \tanh(u_i)$，其中 $c_i = (a_{\max,i} + a_{\min,i})/2$、$b_i = (a_{\max,i} - a_{\min,i})/2$。

10.3 把 Actor Loss 改写为 Pathwise Objective

记 $a = f_\theta(s,\epsilon)$，用固定噪声分布替换动作分布中的随机性:

$$\boxed{J_\pi(\theta) = \mathbb{E}_{\substack{s\sim\mathcal{D} \\ \epsilon\sim\mathcal{N}(0,I)}} \left[\alpha \log \pi_\theta(f_\theta(s,\epsilon) \mid s) - Q_\phi(s, f_\theta(s,\epsilon))\right].} \tag{19}$$

令 $a_\theta = f_\theta(s,\epsilon)$，对参数求全导数:

$$\boxed{\nabla_\theta J_\pi = \mathbb{E}\left[\alpha \frac{d}{d\theta} \log \pi_\theta(a_\theta \mid s) - \nabla_a Q_\phi(s,a_\theta) \frac{\partial a_\theta}{\partial \theta}\right].} \tag{20}$$

注意 $\log \pi_\theta(a_\theta \mid s)$ 既显式依赖 θ，又通过 a_θ 隐式依赖 θ:

$$\frac{d}{d\theta} \log \pi_\theta(a_\theta \mid s) = \left.\frac{\partial \log \pi_\theta(a \mid s)}{\partial \theta}\right|_{a=a_\theta} + \nabla_a \log \pi_\theta(a_\theta \mid s) \frac{\partial a_\theta}{\partial \theta}.$$

故:

$$\boxed{\nabla_\theta J_\pi = \mathbb{E}\left[\alpha \left.\frac{\partial \log \pi_\theta(a \mid s)}{\partial \theta}\right|_{a=a_\theta} + \left(\alpha \nabla_a \log \pi_\theta(a_\theta \mid s) - \nabla_a Q_\phi(s, a_\theta)\right) \frac{\partial a_\theta}{\partial \theta}\right].} \tag{21}$$

其中最具有控制意义的部分是：

$$\underbrace{\nabla_a Q_\phi(s, a_\theta)}_{\text{动作朝哪个方向更好}} \qquad \underbrace{\frac{\partial a_\theta}{\partial \theta}}_{\text{如何改变 Actor 参数}}.$$

这使 SAC 与 DDPG 的确定性策略梯度有直接联系：SAC 在可重参数化的**随机策略**中利用 Critic 的动作梯度，并额外进行熵正则化。Pathwise Gradient 通常比直接使用 Score-Function Estimator 具有更低的方差，但效果仍依赖 Critic 的质量。

11. 第十步：Tanh Gaussian 的 Log Probability 修正

Gaussian 变量 u 与 $a = \tanh(u)$ 的分布不同，因此不能直接使用 $\log \mathcal{N}(a; \mu, \Sigma)$。对可逆变换 $a = g(u)$，由密度变换：

$$\pi(a \mid s) = p_U(u \mid s) \left|\det \frac{\partial a}{\partial u}\right|^{-1},$$

$$\log \pi(a \mid s) = \log p_U(u \mid s) - \log \left|\det \frac{\partial a}{\partial u}\right|.$$

对于逐维 $a_i = \tanh(u_i)$，有 $\partial a_i / \partial u_i = 1 - \tanh^2(u_i)$。Jacobian 是对角矩阵，其行列式为 $\prod_i (1 - \tanh^2(u_i))$。所以：

$$\boxed{\log \pi_\theta(a \mid s) = \log \mathcal{N}(u; \mu_\theta(s), \Sigma_\theta(s)) - \sum_{i=1}^{d} \log\left(1 - \tanh^2(u_i)\right).} \tag{22}$$

如果将动作进一步缩放为 $a_i^{\text{env}} = c_i + b_i \tanh(u_i)$，且 $b_i > 0$：

$$\boxed{\log \pi_\theta(a^{\text{env}} \mid s) = \log \mathcal{N}(u; \mu_\theta, \Sigma_\theta) - \sum_{i=1}^{d} \log\left(b_i(1 - \tanh^2(u_i))\right).} \tag{23}$$

实现时为避免 $\tanh(u)$ 接近 ± 1 引起的数值误差，可用稳定恒等式 $\log(1 - \tanh^2 u) = 2(\log 2 - u - \text{softplus}(-2u))$，或者加上数值稳定项。常见实现先在归一化动作空间计算 Log Probability，再将动作缩放给环境；重要的是整个熵与动作尺度约定保持一致。

动机总结：SAC 的 Actor 和温度调节都依赖 $\log \pi$；漏掉 Tanh 的 Jacobian Correction 会直接改变优化目标和梯度方向。

12. 第十一步：Automatic Entropy Tuning

12.1 从固定温度到熵约束

手工设定 α 对奖励缩放和动作维度敏感。与其直接选择熵正则化系数，不如要求平均策略熵达到目标水平。一个常用的概括形式为：

$$\boxed{\begin{aligned}&\max_{\pi} \quad J_R(\pi)\\ &\text{s.t.} \quad \bar{\mathcal{H}}(\pi) \geq \mathcal{H}_{\text{target}}.\end{aligned}} \tag{24}$$

$\bar{\mathcal{H}}$ 表示相关状态访问分布上的平均策略熵。原论文以逐时刻熵约束推导温度学习；这里采用等价设计思想的平均熵表述来简化解释。

引入非负乘子 α，拉格朗日函数为：

$$\boxed{\mathcal{L}(\pi, \alpha) = J_R(\pi) + \alpha\big(\bar{\mathcal{H}}(\pi) - \mathcal{H}_{\text{target}}\big).} \tag{25}$$

相应形式是：

$$\boxed{\max_{\pi} \min_{\alpha \geq 0} \mathcal{L}(\pi, \alpha).} \tag{26}$$

固定 α 优化 π，就回到最大熵策略优化；固定策略优化 α，则得到熵目标的对偶更新。

12.2 温度损失及其更新方向

用 $\mathcal{H}(\pi) = -\mathbb{E} \log \pi$，得：

$$\boxed{J_\alpha(\alpha) = \mathbb{E}_{s\sim\mathcal{D}, a\sim\pi_\theta}\left[\alpha\left(-\log \pi_\theta(a \mid s) - \mathcal{H}_{\text{target}}\right)\right].} \tag{27}$$

等价地 $J_\alpha = -\mathbb{E}[\alpha(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}})]$。求导：

$$\boxed{\frac{\partial J_\alpha}{\partial \alpha} = \bar{\mathcal{H}}(\pi_\theta) - \mathcal{H}_{\text{target}}.} \tag{28}$$

所以：

$$\boxed{\begin{cases} \bar{\mathcal{H}}(\pi) < \mathcal{H}_{\text{target}} & \Rightarrow \alpha \uparrow, \\ \bar{\mathcal{H}}(\pi) > \mathcal{H}_{\text{target}} & \Rightarrow \alpha \downarrow. \end{cases}} \tag{29}$$

熵偏低时，提高 α 让 Actor 更重视探索；熵偏高时，降低 α 让 Actor 更集中于高价值动作。

12.3 为什么学习 Log-Alpha?

直接更新 α 可能导致负值。因此定义 $\eta = \log \alpha$、$\alpha = e^\eta > 0$。严格代入对偶损失为：

$$J_\eta(\eta) = -\mathbb{E}\left[e^\eta\left(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}}\right)\right].$$

它的梯度包含 $e^\eta = \alpha$。许多常见实现采用梯度尺度不同、但内部驻点相同的代理损失：

$$\boxed{J_\eta^{\text{surrogate}}(\eta) = -\mathbb{E}\left[\eta\left(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}}\right)\right].} \tag{30}$$

温度更新时必须将 Log Probability `detach`，避免温度损失反向更新 Actor。

12.4 Target Entropy 的经验选择

对于 d 维归一化连续动作，常见启发式为 $\mathcal{H}_{\text{target}} = -d$。例如 $d = 12$ 时可先试 -12。这不是理论上的最优公式；连续微分熵允许负值，且会受动作尺度与概率密度约定影响。机器人高维动作、手部关节或动作序列控制往往需要验证这一超参数，而不能机械套用。

13. 第十二步：完整的现代 SAC 算法

13.1 模块与三个优化问题

现代连续动作 SAC 一般包含以下模块：

模块	参数/对象	作用
Actor	θ	生成可重参数化的随机动作
Critic 1 / 2	ϕ_1, ϕ_2	预测 Soft Q，缓解过高估计
Target Critics	$\bar{\phi}_1, \bar{\phi}_2$	产生缓慢变化的 TD Target
Temperature	α 或 $\eta = \log \alpha$	调整熵权重
Replay Buffer	$\mathcal{D}$	保存历史 (s, a, r, s', d)

三个核心目标：

$$\boxed{\min_{\phi_1,\phi_2} J_Q, \qquad \min_{\theta} J_\pi, \qquad \min_{\alpha \geq 0} J_\alpha.}$$

13.2 一次更新的完整步骤

步骤 1：交互并写入 Buffer。由当前状态 s_t 采样 $a_t \sim \pi_\theta(\cdot \mid s_t)$，执行动作，记录 $(s_t, a_t, r_t, s_{t+1}, d_t)$。初始训练阶段可以使用随机动作收集一定的覆盖数据。

步骤 2：历史采样。从 $\mathcal{D}$ 抽取 Batch $B = \{(s_i, a_i, r_i, s_i', d_i)\}_{i=1}^N$。

步骤 3：计算目标。用当前 Actor 在下一状态采样 $a_i' \sim \pi_\theta(\cdot \mid s_i')$，计算：

$$\boxed{y_i = r_i + \gamma(1 - d_i)\left[\min_{j=1,2} Q_{\bar{\phi}_j}(s_i', a_i') - \alpha \log \pi_\theta(a_i' \mid s_i')\right].}$$

步骤 4：更新 Critic。两个 Critic 使用相同目标，分别最小化：

$$J_{Q_j} = \frac{1}{2N}\sum_{i=1}^{N}(Q_{\phi_j}(s_i, a_i) - y_i)^2.$$

步骤 5：更新 Actor。重新采样 $\epsilon_i \sim \mathcal{N}(0, I)$、$\tilde{a}_i = f_\theta(s_i, \epsilon_i)$，最小化：

$$\boxed{J_\pi = \frac{1}{N}\sum_{i=1}^{N}\left[\alpha \log \pi_\theta(\tilde{a}_i \mid s_i) - \min_j Q_{\phi_j}(s_i, \tilde{a}_i)\right].}$$

Actor 更新时冻结 Critic **参数**，但必须允许梯度通过 Critic 输入动作，即保留 $\nabla_a Q_\phi$。

步骤 6：更新温度。根据 $-\log \pi_\theta$ 与目标熵之差调整 α，并在此步骤对策略概率进行 `detach`。

步骤 7：更新目标网络。对每个 Critic 做 $\bar{\phi}_j \leftarrow (1-\tau)\bar{\phi}_j + \tau\phi_j$。然后重复。

13.3 教学用伪代码

```
initialize actor pi_theta
initialize critics Q1, Q2
initialize target critics Q1_targ, Q2_targ from Q1, Q2
initialize replay buffer D and log_alpha

for each environment interaction:
    action = pi_theta.sample(obs)  # warm-up may use random actions
    next_obs, reward, terminated, truncated = env.step(action)
    D.add(obs, action, reward, next_obs, terminated)
    obs = env.reset() if (terminated or truncated) else next_obs

    for each gradient update:
        bs, ba, br, bs_next, bd = D.sample(batch_size)
        alpha = exp(log_alpha)

        # 1) Critic target: no gradient through target branch
        with no_grad():
            a_next, logp_next = pi_theta.rsample(bs_next)
            q_next = min(Q1_targ(bs_next, a_next),
                         Q2_targ(bs_next, a_next))
            y = br + gamma * (1 - bd) * (q_next - alpha * logp_next)

        loss_q1 = mean((Q1(bs, ba) - y) ** 2)
        loss_q2 = mean((Q2(bs, ba) - y) ** 2)
        update(Q1, loss_q1)
        update(Q2, loss_q2)

        # 2) Actor: freeze Q weights, keep dQ/da
        a_pi, logp_pi = pi_theta.rsample(bs)
        q_pi = min(Q1(bs, a_pi), Q2(bs, a_pi))
        loss_pi = mean(alpha.detach() * logp_pi - q_pi)
        update(pi_theta, loss_pi)

        # 3) Automatic entropy tuning (log-alpha surrogate)
        loss_alpha = -mean(log_alpha *
                           (logp_pi.detach() + target_entropy))
        update(log_alpha, loss_alpha)

        # 4) Polyak target update
        for Q, Q_targ in [(Q1, Q1_targ), (Q2, Q2_targ)]:
            Q_targ.params = (1 - tau) * Q_targ.params + tau * Q.params
```

实现警告：上面是为了说明更新顺序而写的伪代码，不是可以直接运行的完整库。真实代码需正确执行优化器零梯度、参数冻结、动作缩放、概率变换、初始探索、环境 Reset 和 Batch Shape

处理；还需要区分 `terminated` 与 `truncated`。

14. SAC v1 与现代 SAC 的区别

14.1 原始 SAC v1：显式训练 V 网络

最初的 SAC 除双 Q 和 Actor 外，还维护状态价值网络 $V_\psi(s)$：

$$V^\pi(s) = \mathbb{E}_{a\sim\pi}\big[Q^\pi(s,a) - \alpha \log \pi(a \mid s)\big].$$

因此可以学习：

$$\boxed{J_V(\psi) = \frac{1}{2}\mathbb{E}_s\left[\left(V_\psi(s) - \left(\min_i Q_{\phi_i}(s,\tilde{a}) - \alpha \log \pi_\theta(\tilde{a} \mid s)\right)\right)^2\right].}$$

目标项在训练 V 时视为常数。其 Q Target 为：

$$\boxed{y_Q = r + \gamma(1-d)V_{\bar{\psi}}(s').}$$

14.2 现代 SAC：去掉显式 V 网络

由于：

$$V^\pi(s') = \mathbb{E}_{a'\sim\pi}\big[Q^\pi(s',a') - \alpha \log \pi(a' \mid s')\big],$$

可直接用下一动作采样近似：

$$V^\pi(s') \approx \min_j Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi_\theta(a' \mid s').$$

这样 Q Target 直接写成式 (12)，不需要单独的 V 网络，减少额外拟合误差及网络复杂度。现在提及“标准 SAC”，通常是指该现代双 Q 版本，并经常搭配自动温度调节。

15. SAC 与 PPO 的本质区别

15.1 策略更新公式对比

PPO 使用采样动作的 Advantage 及策略概率比：

$$\boxed{J_{\text{PPO}}(\theta) = \mathbb{E}\left[\min\left(\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right],}$$

其中 $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$。优化器最大化该目标，并通过 Clipping 抑制剧烈的策略变化。典型策略梯度由 $\nabla_\theta \log \pi_\theta(a \mid s)\, A^\pi(s,a)$ 形成。

SAC 则最小化：

$$\boxed{J_{\text{SAC}}(\theta) = \mathbb{E}\left[\alpha \log \pi_\theta(a_\theta \mid s) - Q_\phi(s,a_\theta)\right], \quad a_\theta = f_\theta(s,\epsilon).}$$

其中包含 $\nabla_a Q_\phi(s,a_\theta)\, \partial a_\theta/\partial\theta$。PPO 通常无需 $\nabla_a Q$；SAC 标准连续 Actor 更新需要它。

15.2 对比表

对比维度	PPO	SAC
数据范式	On-Policy	Off-Policy
Actor	通常随机策略	可重参数化随机策略
Critic	常用 $V(s)$	常用 $Q(s,a)$
策略优化信号	Advantage、Probability Ratio	Q 值、动作梯度、熵
主要梯度估计	Score-Function	Pathwise / Reparameterization
历史数据复用	有限，主要使用新 Rollout	Replay Buffer 可重复采样
熵项	常用但非必需	核心目标
稳定机制	Clipped Policy Update	Double Q、Target Critics
样本效率	通常较低	通常较高
典型优势	大规模并行仿真吞吐量高	环境交互成本昂贵时可充分复用数据
主要挑战	新样本需求大	Critic 外推、高估和 Bootstrap 稳定性

PPO Clipping 与 SAC 的 Double Q **不是同一类机制**：前者主要限制策略概率变化，后者主要缓解 Critic 高估；SAC 没有标准 PPO 式的概率比 Clipping。

15.3 为什么机器人仿真中 PPO 仍常见?

在 Isaac Lab 等平台上，大规模并行仿真可以以较低单位成本生成大量轨迹。PPO 通过高吞吐的并行 Rollout 获得出色的训练速度；SAC 虽复用样本，但 Critic Bootstrapping、频繁梯度更新和 Replay Buffer 访问也有成本。因此：

$$\boxed{\text{Sample Efficiency} \neq \text{Wall-Clock Efficiency.}}$$

高接触非线性、稀疏奖励、状态分布偏移也会增加 SAC 的价值学习难度。在真实机器人环境交互昂贵时，SAC 的 Off-Policy 数据复用通常更具吸引力，但仍需考虑安全探索和离线数据偏移。

16. 对灵巧手与 Flow Matching Policy 的启示

16.1 机器人连续动作的局部优化

灵巧操作可以采用状态 $s = (q, \dot{q}, T_{\text{object}}, \ldots)$，动作 $a = (\Delta q_{\text{arm}}, \Delta q_{\text{hand}})$。Critic 估计 $Q_\phi(s,a)$；Actor 利用 $\nabla_a Q_\phi(s,a)$ 改善动作。在动作价值较平滑、Critic 估计准确时，这提供了有效的局部优化方向。

接触操作可能含接触模式切换、碰撞和摩擦等非光滑过程，神经网络的 $\nabla_a Q$ 未必真实反映物理系统的局部变化；因此学习到的梯度质量往往是关键瓶颈。

16.2 冻结预训练动作生成器，优化输入噪声

考虑预训练可微动作生成器:

$$a = f_{\text{pretrained}}(s, z).$$

冻结生成器，仅学习噪声分布 $z \sim \pi_\theta(z \mid s)$，可设计 Latent-Space Maximum-Entropy Actor-Critic 目标:

$$\boxed{J(\theta) = \mathbb{E}_{s,z}\left[\alpha \log \pi_\theta(z \mid s) - Q_\phi(s, f_{\text{pretrained}}(s,z))\right].} \tag{31}$$

如果生成器可微，梯度包含：

$$\boxed{\nabla_a Q_\phi(s,a)\,\frac{\partial f_{\text{pretrained}}(s,z)}{\partial z}\,\frac{\partial z_\theta}{\partial \theta}.}$$

这可以解释“冻结大生成器、只训练较小的噪声 Actor”的一种理论路径。不过 $\mathcal{H}(\pi(z \mid s))$ 是**潜变量熵**，并不自动等于最终动作分布的熵 $\mathcal{H}(\pi(a \mid s))$；只有满足适当可逆性及密度变换条件时才能进一步关联。因此，这与动作空间标准 SAC 不可无条件等同。

16.3 为什么 Flow Matching Policy 直接接 SAC 更困难?

Flow Matching 用连续速度场生成动作：

$$\frac{dx_t}{dt} = v_\theta(x_t, s, t), \qquad x_0 \sim p_0, \quad a = x_1.$$

若直接套用 SAC，需要：

$$J(\theta) = \mathbb{E}\left[\alpha \log \pi_\theta(a \mid s) - Q_\phi(s,a)\right].$$

但动作来自速度场数值积分，梯度可能需要穿过 ODE 求解；精确动作 Log Density 可能需要连续变量变换以及 Jacobian Trace 的计算；对大规模生成模型会带来时间与内存成本。Flow Matching **并非理论上无法**计算密度，而是通常比 Squashed Gaussian 更复杂。

由此产生几类研究路径：优化低维初始噪声分布、训练 Residual Policy、使用 Q-Guided 或 Advantage-Guided 的速度场适配，以及构造不显式依赖动作 Log Probability 的替代优化目标。这些方法都希望把价值信息反馈到生成策略，但对应的损失函数与约束必须分别推导。

17. 最终总结：五组关键公式与理解框架

（1）Maximum-Entropy Objective：

$$\boxed{J(\pi) = \mathbb{E}_\tau\left[\sum_{t=0}^{\infty} \gamma^t (r_t - \alpha \log \pi(a_t \mid s_t))\right].}$$

动机：最大化回报，同时通过熵正则化维持探索。

（2）Soft Bellman Equation：

$$\boxed{Q^\pi(s,a) = \mathbb{E}\left[r + \gamma(Q^\pi(s',a') - \alpha \log \pi(a' \mid s'))\right].}$$

动机：将最大熵目标递归分解，使 Critic 能进行 TD 学习。

（3）KL-Based Soft Policy Improvement：

$$\boxed{\pi_{\text{new}} = \arg\min_\pi D_{\text{KL}}\left(\pi(\cdot \mid s) \,\middle\|\, \frac{\exp(Q^{\pi_{\text{old}}}(s,\cdot)/\alpha)}{Z(s)}\right),}$$

等价于最小化 $\mathbb{E}_{a\sim\pi}[\alpha \log \pi(a \mid s) - Q^{\pi_{\text{old}}}(s,a)]$。动机：逼近高价值 Boltzmann 分布而不计算归一化常数。

（4）现代 SAC 的 Critic 与 Actor：

$$\boxed{J_{Q_i} = \frac{1}{2}\mathbb{E}_{\mathcal{D}}\big[(Q_{\phi_i}(s,a) - y)^2\big], \quad y = r + \gamma(1-d)\big[\min_j Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi_\theta(a' \mid s')\big],}$$

$$\boxed{J_\pi = \mathbb{E}_{s,\epsilon}\big[\alpha \log \pi_\theta(f_\theta(s,\epsilon) \mid s) - \min_j Q_{\phi_j}(s, f_\theta(s,\epsilon))\big].}$$

动机：用可训练网络、历史数据、双 Q 和 Pathwise Gradient 实现 Soft Policy Iteration。

（5）Automatic Entropy Tuning:

$$\boxed{J_\alpha = -\mathbb{E}\big[\alpha(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}})\big].}$$

动机：根据目标熵自动调节探索强度。

最关键的 PPO / SAC 区别

PPO 主要通过 $\nabla_\theta \log \pi_\theta(a \mid s) A^\pi(s,a)$ 调整**已采样动作的概率**；连续 SAC 使用 $\nabla_a Q_\phi(s,a)\, \partial a_\theta / \partial \theta$（同时有熵梯度），调整**可微动作生成函数**以提高动作价值。

完整逻辑链条为：

$$\boxed{\begin{array}{c}\text{Maximum-Entropy RL} \longrightarrow \text{Soft Bellman} \longrightarrow \text{KL Policy Improvement} \\ \longrightarrow \text{Reparameterization} \longrightarrow \text{Off-Policy Actor-Critic}\end{array}}$$

Double Q、Target Critics 和 Automatic Temperature Tuning 则使上述理论更加适合复杂连续控制任务。

参考资料

1. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). **Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.** ICML. https://arxiv.org/abs/1801.01290 —Soft Policy Iteration、KL 策略改进和随机 Actor 的理论基础。
2. Haarnoja, T., Zhou, A., Hartikainen, K., et al. (2018). **Soft Actor-Critic Algorithms and Applications.** https://arxiv.org/abs/1812.05905 —自动温度调节和现代 SAC 设计。
3. OpenAI Spinning Up. **Soft Actor-Critic.** https://spinningup.openai.com/en/latest/algorithms/sac.html —现代 SAC 目标函数与工程实现。

文档说明：本 *Markdown* 与其同名 *PDF* 使用同一正文源文件生成；章节、公式、表格与参考资料保持一致。